

THRESHOLD ENTERPRISE SYSTEMS

API Development Guidelines

Document Id	ENG-001
Title	API Development Guidelines
Department	Engineering
Classification	INTERNAL
Version	1.0
Status	ACTIVE
Effective Date	2026-01-15
Document Type	GUIDELINE
Allowed Roles	EMPLOYEE, MANAGER, ENGINEER, AI_ENGINEER, SECURITY_ENGINEER, OPERATIONS_MANAGER, ADMIN

1. Executive Summary and Architectural Purpose

This document establishes the mandatory architectural standards, design conventions, security principles, and testing requirements governing all Application Programming Interfaces (APIs) designed, implemented, and maintained across Threshold Enterprise Systems ('Threshold').

As an enterprise technology provider operating distributed cloud services, AI governance platforms, and multi-tenant SaaS products across India, the United States, the United Kingdom, and Singapore, APIs form the foundational connective tissue of our digital ecosystem. Inconsistent API contracts, unpredictable error structures, inadequate validation, and loose versioning introduce compounding technical debt, operational friction, and security vulnerabilities.

The purpose of these API Development Guidelines is to provide software engineers, AI engineers, and integration specialists with an authoritative, practical framework for creating robust, maintainable, performant, and secure RESTful APIs. Adherence ensures seamless interoperability across microservices, simplifies client SDK generation, enforces enterprise zero-trust security baselines, and guarantees consistent developer experience (DX) across all engineering squads.

2. Scope and Engineering Applicability

These guidelines apply universally to all backend engineers, full-stack developers, AI pipeline engineers, data engineers, and third-party development contractors building software for Threshold Enterprise Systems.

The scope encompasses:

- Internal Microservice APIs: Private REST and gRPC interfaces facilitating synchronous and asynchronous communication within enterprise service meshes.
- Customer-Facing Public APIs: External commercial endpoints exposed to enterprise clients, external developer portals, and third-party SaaS integrations.
- AI Model Serving and Inference Endpoints: Specialized HTTP/REST and streaming WebSocket APIs exposing machine learning models, retrieval-augmented generation (RAG) query engines, and vector search services.
- Administrative and Operational Endpoints: Internal management APIs utilized by DevOps tooling, CI/CD orchestration, monitoring daemons, and database maintenance scripts.

3. Core API Design Principles and REST Conventions

All enterprise APIs engineered at Threshold must strictly embody seven foundational design principles:

Principle 1 - Resource-Oriented Architecture: APIs must model domain entities as nouns rather than verbs. URIs must represent resources, while HTTP verbs declare the operational intent applied to those resources.

Principle 2 - Predictable URI Naming Standards: URIs must use lowercase alphanumeric characters with hyphens for multi-word segments (kebab-case). Collections must be pluralized (e.g., `/api/v1/governance-policies`, `/api/v1/model-evaluations`). Deeply nested hierarchies must be avoided; nesting should not exceed two levels (e.g., `/api/v1/workspaces/{workspace_id}/documents`).

Principle 3 - Strict HTTP Verb Semantics: HTTP methods must adhere to RFC 9110 standards:

- GET: Idempotent and safe retrieval of resources; must never cause side effects or modify persistent state.
- POST: Creation of subordinate resources or execution of complex non-idempotent processing tasks.
- PUT: Idempotent full replacement of an existing resource with the submitted representation.
- PATCH: Partial atomic update of specified resource attributes.
- DELETE: Idempotent removal of the specified resource.

Principle 4 - Explicit Content Negotiation: All endpoints must explicitly declare and validate media types. Requests and responses must exchange data via `application/json` using standard UTF-8 character encoding.

Principle 5 - Stateless Operation: Every HTTP request must encapsulate all necessary authentication tokens, contextual headers, and parameters required for isolated processing without relying on server-side HTTP session state.

Principle 6 - Graceful Backward Compatibility: Existing API contracts must not introduce breaking schema modifications within a major version.

Principle 7 - Comprehensive Machine-Readable Documentation: Every production API must publish an up-to-date OpenAPI Specification (OAS 3.1) generated automatically from typed backend schemas.

4. Request Validation, Input Sanitization, and Parameter Handling

Input validation is the first line of defense against injection attacks, data corruption, and catastrophic system failure:

- Declarative Schema Validation: All Python backend services (e.g., FastAPI, Pydantic) must enforce strict declarative data modeling. Every incoming request payload must be parsed against strongly typed models defining data types, string length bounds, numeric ranges, and regex patterns.
- Extra Field Rejection: Request schemas must configure model configurations to forbid or discard unparsed arbitrary fields (e.g., `Pydantic extra='forbid'`) to prevent mass assignment vulnerabilities.
- Query Parameter Conventions: Query parameters must use `snake_case` naming. Standardized parameter keywords include: `limit` and `offset` for pagination, `sort_by` and `order` (`asc/desc`) for sorting, and `q` for generic keyword search.
- Path Parameter Validation: Resource identifiers embedded in URL paths must enforce UUIDv4 or strongly validated integer constraints, rejecting malformed tokens prior to business logic execution.

5. Standardized Response Structures and Envelope Conventions

Consistency in API response formatting accelerates client integration and ensures reliable error handling:

- Successful Single-Resource Representation: Successful GET, POST, or PUT operations return the resource object directly or within a standardized payload structure with HTTP 200 (OK) or HTTP 201 (Created).
- Standardized Collection Response: List endpoints must return an array accompanied by pagination metadata:

```
{ "data": [...], "pagination": { "total_count": 1250, "limit": 50, "offset": 0, "has_more": true } }
```

- Cursoring for High-Volume Streaming: For high-throughput event streams or AI telemetry logs exceeding 100,000 items, cursor-based pagination using opaque base64-encoded continuation tokens (e.g., `next_cursor`) must be implemented instead of offset pagination.
- Empty Collection Handling: When a query yields zero matching records, the API must return HTTP 200 with an empty JSON array (`[]`), never HTTP 404.

6. Enterprise Error Handling and HTTP Status Code Matrix

APIs must communicate failure conditions using appropriate HTTP status codes paired with machine-readable RFC 7807 problem details:

- Standard Problem Details Schema: All 4xx and 5xx responses must return a structured JSON object containing: `error_code`, `message`, `details` (array of field-level validation errors), `timestamp`, and `request_id` (for distributed tracing).

- Client Error Codes (4xx):

- * 400 Bad Request: Malformed JSON syntax or schema validation failure.
- * 401 Unauthorized: Missing, expired, or invalid cryptographic bearer token.
- * 403 Forbidden: Authenticated user lacks authorized role permissions under SEC-002.
- * 404 Not Found: Target resource URI does not exist.
- * 409 Conflict: State collision, duplicate unique key, or concurrent modification conflict.
- * 422 Unprocessable Entity: Syntactically valid JSON that violates domain business rules.
- * 429 Too Many Requests: Client exceeded rate-limiting token quota.

- Server Error Codes (5xx):

- * 500 Internal Server Error: Unhandled backend exception (details must be masked from client).
- * 502 Bad Gateway: Downstream dependency or vector store connection failure.
- * 503 Service Unavailable: Temporary capacity exhaustion or scheduled maintenance window.
- * 504 Gateway Timeout: Downstream microservice exceeded execution latency budget.

7. Security, Authentication, and Permission-Aware Integration

APIs must enforce zero-trust security controls in compliance with SEC-002 (Access Control Policy) and SEC-005 (Data Classification and Handling Policy):

- Cryptographic Token Validation: All endpoints must mandate OAuth 2.0 / OpenID Connect JWT bearer tokens signed by enterprise identity providers using asymmetric RS256/ES256 keys.
- Role-Based Access Control (RBAC): Endpoint handlers must inspect the user's authenticated roles (EMPLOYEE, MANAGER, ENGINEER, AI_ENGINEER, SECURITY_ENGINEER, OPERATIONS_MANAGER, HR_MANAGER, ADMIN) and reject unpermitted operations with HTTP 403.
- Data Masking and Least Privilege: Endpoints returning database records must filter fields based on caller permissions (e.g., masking employee salary attributes in HR-007 from non-HR personnel).
- Database Connection Security: Backend APIs communicating with persistent relational or vector stores must utilize dedicated service accounts governed by the Database Access Policy (ENG-005), enforcing SSL/TLS encryption.

8. API Versioning, Deprecation, and Backward Compatibility

To ensure stability across distributed client applications and microservices, strict lifecycle governance applies:

- URI Path Versioning: Major API revisions must be declared in the URI prefix (e.g., /api/v1/, /api/v2/). Minor and patch enhancements that maintain backward compatibility must not increment the path version.
- Backward Compatibility Mandate: Adding optional request fields, introducing new response fields, or optimizing internal algorithms are permitted within a major version. Removing fields, renaming attributes, or changing data types constitutes a breaking change requiring a new major version.
- Deprecation Protocol: When an API version is targeted for retirement, engineering teams must provide a minimum of six (6) months advance notice. Deprecated endpoints must emit Sunset and Deprecation HTTP response headers specifying the scheduled decommission date.

9. Observability, Structured Logging, and Distributed Tracing

Production APIs must provide comprehensive telemetry to support operational monitoring and incident triage:

- Distributed Trace Header Propagation: APIs must propagate W3C Trace Context headers (traceparent, tracestate) across microservice boundaries, enabling distributed tracing in OpenTelemetry.
- Structured JSON Logging: All log emissions must output structured JSON containing timestamp, service_name, environment, trace_id, span_id, http_method, request_uri, status_code, and latency_ms.
- PII and Credential Masking in Logs: In strict adherence to SEC-005, log handlers must automatically redact Authorization headers, API keys, session cookies, and personal identity tokens.

10. API Testing, Quality Gates, and Governance References

Before promotion to production, every API must pass rigorous automated testing pipelines:

- Automated Unit and Integration Testing: Test suites must achieve a minimum of 85% statement coverage, verifying positive scenarios, edge-case boundaries, and error condition handlers.
- Contract and Schema Testing: Automated validation verifying that live responses conform 100% to published OpenAPI specifications.

- Cross-Document Governance Interconnections:

- * SEC-002: Access Control Policy (Establishes identity, authentication, and RBAC standards).
- * SEC-005: Data Classification and Handling Policy (Governs sensitive data field handling).
- * ENG-002: Code Review Policy (Enforces API contract review during pull requests).
- * ENG-005: Database Access Policy (Regulates backend database connection pooling and queries).